

CSC-207L Advanced Database Management Systems

Adaptive Buffer management system

Project Report



Submitted By:

Syed Shayan Arshad	2023-CS-625
Adeel-Ur-Rehman	2023-CS-626
Hammad Khalid	2023-CS-637

Submitted To:

Mam. Rida

Dated: 27th April 2025

Department of Computer Science

University of Engineering and Technology Lahore, New Campus

Introduction

This project implements a Buffer Management System to simulate and evaluate buffer replacement policies such as Least Recently Used (LRU), Most Recently Used (MRU), and Least Frequently Used (LFU). Built using Flask, Python, and SQL Server, the system dynamically switches policies based on workload patterns and benchmarks their performance through hit/miss ratios and eviction statistics. The primary goal is to analyze how different policies perform under various workloads (e.g., read-heavy, write-heavy, repeated, random) and optimize caching efficiency. The system uses a database with some pages and allows users to specify the number of iterations and buffer size for simulations.

Theoretical Basis of Buffer Replacement Policies

Least Recently Used (LRU)

The LRU policy evicts the least recently accessed page when the buffer is full. It operates on the principle of temporal locality, assuming that pages accessed recently are more likely to be accessed again soon. For example, in a web browser cache, LRU ensures that recently visited pages remain available for quick access.

Advantages: LRU is highly effective for workloads with repetitive access patterns, as it retains pages that are likely to be reused. It performs well in scenarios like database query caching, where recently accessed data is often queried again.

Disadvantages: LRU requires maintaining a list of pages in order of their access time, which introduces computational overhead, especially for large buffers. Additionally, it may evict pages that are still relevant if they haven't been accessed recently.

Most Recently Used (MRU)

The MRU policy evicts the most recently accessed page. It is based on the assumption that the most recently used page is less likely to be accessed again in certain scenarios, such as sequential scans in databases where pages are accessed once and then not needed again.

Advantages: MRU is simple to implement and can be effective in specific workloads, such as one-time data processing tasks where recently used pages are no longer needed. It reduces the overhead of tracking long-term access patterns.

Disadvantages: MRU is less effective in scenarios with repetitive access patterns, as it may evict pages that are likely to be reused soon. Its applicability is limited compared to LRU or LFU, making it less commonly used in general caching systems.

Least Frequently Used (LFU)

The LFU policy evicts the page with the lowest access frequency when the buffer is full. It prioritizes pages that are accessed frequently over time, assuming that frequently used pages will continue to be relevant. For example, in a file server cache, LFU ensures that frequently accessed files remain available.

Advantages: LFU is effective for workloads with stable access patterns, where certain pages are consistently accessed more often. It performs well in scenarios like content delivery networks, where popular content is accessed repeatedly.

Disadvantages: LFU requires tracking the access frequency of each page, which increases memory and computational overhead. It can also retain stale pages that were frequently accessed in the past but are no longer relevant, leading to potential inefficiencies.

These policies address different caching needs, and their performance depends on the workload characteristics, as discussed in the next section.

Tuning Based on Workload

The Buffer Management System dynamically tunes its replacement policy based on workload patterns to optimize caching performance. The system supports four workload types: read-heavy, write-heavy, repeated, and random, each simulating distinct access patterns:

Read-Heavy: Dominated by read operations with repetitive page accesses. LRU is expected to perform well here due to its focus on retaining recently accessed pages.

Write-Heavy: Involves frequent write operations, often introducing new pages. MRU may be more suitable as it evicts recently written pages, which are less likely to be accessed again soon.

Repeated: Features high repetition of specific pages, where LFU excels by retaining frequently accessed pages.

Random: Combines read and write operations with no clear pattern, requiring dynamic policy switching to adapt to changing access behaviors.

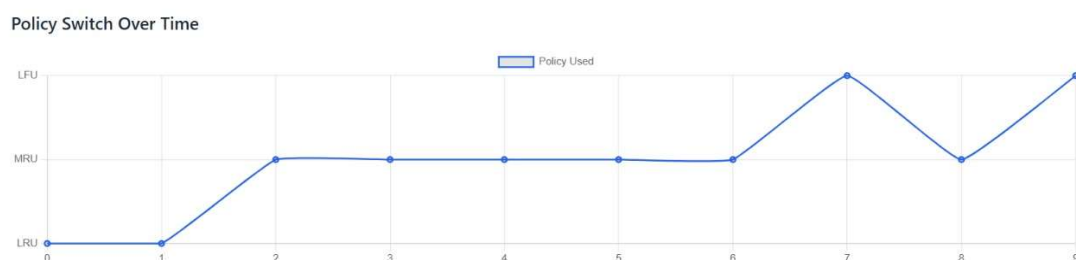
Relevant Graphs

The Buffer Management System generates graphs to analyze performance, using data from policy data and stats. The graphs below were created with 48 iterations and a cache size of 3, using a random workload to reflect varied access patterns.

Policy Switch Over Time (Line Graph)

Description: This line graph plots policy changes over 48 operations, with the y-axis showing the policy (0=LRU, 1=MRU, 2=LFU) and the x-axis showing operation numbers. The graph tracks how the system switches between LRU, MRU, and LFU every 5 operations.

Result: The graph starts with LRU (0), moves to MRU (1) at operation 1, remains at MRU until operation 5, then fluctuates between MRU and LFU (2) from operation 6 onwards, peaking at LFU at operations 8 and 9. This shows the system's dynamic policy switching to adapt to workload patterns.

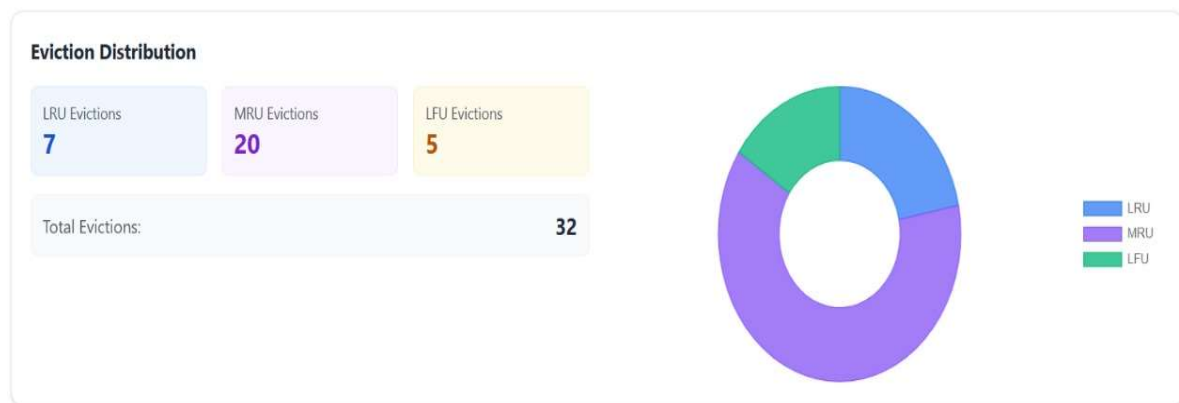


Cache Hit/Miss Ratio (Pie Chart)

Description: This pie chart displays the hit/miss ratio for a run with 48 iterations and a cache size of 3, with hits in green and misses in red, representing the percentage of cache hits and misses during the simulation.

Result: The chart shows 13 hits (27.1%) and 35 misses (72.9%) out of 48 operations. The low hit rate reflects the random workload's diverse page accesses, causing frequent misses due to the small cache size of 3, which struggles to retain frequently accessed pages.

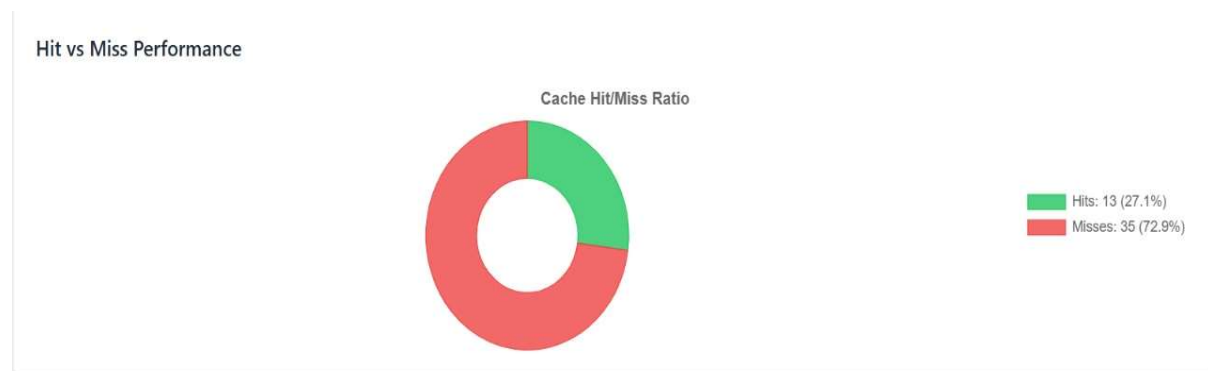
Eviction Analysis



Eviction Analysis (Pie Chart)

Description: This pie chart, along with stats, breaks down evictions by policy (LRU, MRU, LFU) for the same run (48 iterations, cache size 3). The stats show eviction counts: 7 for LRU, 20 for MRU, and 5 for LFU, totalling 32 evictions.

Result: MRU dominates with 20 evictions, followed by LRU with 7 and LFU with 5. The high MRU evictions align with its frequent use in the policy switch graph, indicating its aggressive eviction strategy. This analysis highlights how policy switches impact buffer management with a small cache size.



These graphs provide insights into policy switching, hit/miss performance, and eviction distribution, confirming the system's ability to adapt to workload changes with a limited cache size.

Conclusion

The Buffer Management System successfully integrates LRU, MRU, and LFU replacement policies and dynamically switches between them based on workload patterns. Performance is evaluated through cache hit/miss ratios and eviction statistics, demonstrating the system's adaptability and efficiency. Different policies perform better under different access behaviors, highlighting the need for flexible strategies. The dynamic switching mechanism improves overall cache performance by responding to changing workload conditions